Machine Learning
Semester 2026-III
Workshop No. 4 — Autoencoders & Variational Autoencoders

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the fourth workshop of the *Machine Learning* course project!

Your team has explored supervised and reinforcement learning approaches. This workshop introduces unsupervised learning methods that learn data structure without explicit labels.

This workshop focuses on developing and analyzing unsupervised models — autoencoders and variational autoencoders — that learn compressed representations of data through reconstruction tasks. These learned representations may reveal structure invisible to supervised methods, complement your earlier models, or enable new capabilities like anomaly detection.

The central question for this workshop is: **What structure do unsupervised models reveal about your dataset, and how can these representations enhance your understanding and improve downstream tasks?**

General Workshop Definition: Autoencoders and variational autoencoders are unsupervised neural network architectures that learn compact latent representations of high-dimensional data by training an encoder-decoder pair to minimize reconstruction error. The VAE extends this by constraining the latent space to follow a prior distribution ($\mathtt{N}(0,\mathbf{I})$), enabling meaningful interpolation and sampling. In this workshop, teams implement both architectures on their assigned dataset, analyze the learned representations through visualization and interpretation, and explore applications such as anomaly detection or data augmentation.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **Standard Autoencoder:** Design and train an autoencoder to reconstruct data from your assigned dataset; analyze reconstruction quality and latent space structure.
- **Variational Autoencoder (`VAE`):** Extend the autoencoder with the reparameterization trick and `KL` divergence regularization; compare latent space structure against the standard autoencoder.
- **Latent Space Analysis:** Visualize the latent space using dimensionality reduction techniques (`t-SNE`, `UMAP`); assess what structure the model discovers without label supervision.
- **Reconstruction-Based Applications:** Explore applications such as anomaly detection (where high reconstruction error indicates anomalies), data augmentation, or data denoising.
- **Representation Comparison:** Compare the unsupervised latent representations from the autoencoders with representations from supervised methods in Workshop 2, reflecting on what each approach captures.

Theoretical Background and References

1. **Autoencoder Foundations:**
   - Hinton & Salakhutdinov (2006), "*Reducing the Dimensionality of Data with Neural Networks*", *Science* — demonstrates that deep autoencoders with layer-wise pretraining produce compact, semantically meaningful representations.
   - Vincent et al. (2010), "*Stacked Denoising Autoencoders*", *JMLR* — introduces noise as a regularizer; training to reconstruct clean data from corrupted inputs produces robust features. Consider using denoising as an optional extension.
   - Goodfellow et al. (2016), *Deep Learning*, Chapter 14 (Autoencoders) — comprehensive treatment of reconstruction loss, regularized autoencoders, and connections to PCA.
2. **Variational Autoencoders:**
   - Kingma & Welling (2013), "*Auto-Encoding Variational Bayes*", *ICLR 2014* — the original `VAE` paper. Derives the `ELBO` loss (`reconstruction loss + KL divergence`) and the reparameterization trick. **Required reading.**
   - Doersch (2016), "*Tutorial on Variational Autoencoders*", *arXiv:1606.05908* — accessible tutorial with clear derivations and implementation guidance.
   - Higgins et al. (2017), "*β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*", *ICLR 2017* — introduces the β coefficient to control the trade-off between reconstruction quality and latent space disentanglement.

3. **Autoencoders for Anomaly Detection:**
    - Chalapathy et al. (2018), "*Deep Learning for Anomaly Detection: A Survey*", *arXiv:1801.04956* — comprehensive overview of anomaly detection with deep learning, including reconstruction-error-based methods.
    - Schlegl et al. (2017), "*Unsupervised Anomaly Detection with Generative Adversarial Networks*", *IPMI 2017* — demonstrates reconstruction and latent-space residuals as anomaly scores.
4. **Latent Space Visualization Tools:**
    - **`t-SNE`:** Van der Maaten & Hinton (2008). Use `sklearn.manifold.TSNE` with appropriate perplexity and iteration counts to reduce learned latent vectors to 2D.
    - **`UMAP`:** McInnes et al. (2018). Install with `pip install umap-learn`. Typically faster than `t-SNE` and better preserves global structure.

Required Artifacts and Methodology

1. **Standard Autoencoder (AE) Architecture:**
    - *Encoder:* Design an encoder network appropriate to your data type. For tabular data: stacked fully-connected layers with decreasing hidden units. For image data: convolutional layers with downsampling. For sequential data: recurrent layers.
    - *Bottleneck:* Project to a latent dimension (suggest $z_\text{dim} \in \{32, 64, 128\}$ depending on data complexity). Experiment with different latent dimensions.
    - *Decoder:* Mirror the encoder to reconstruct the original data. Output activations should match data type (e.g., `Tanh` for images in $[-1, 1]$, linear for regression targets).
    - *Loss:* Mean Squared Error (`MSE`) or other reconstruction loss appropriate to your data.
    - Train for 30–50 epochs with `Adam` optimizer. Log training and validation reconstruction loss per epoch.
2. **Variational Autoencoder (VAE) Architecture:**
    - *Encoder:* Same as the standard AE, but the bottleneck produces two outputs: mean $\mu \in \mathbb{R}^{z_\text{dim}}$ and log-variance $\log \sigma^2 \in \mathbb{R}^{z_\text{dim}}$.
    - *Reparameterization:* $z = \mu + \sigma \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. This makes the sampling operation differentiable.
    - *Decoder:* Same as the standard AE, taking z as input.
    - *Loss (`ELBO`):* $\mathcal{L} = \mathcal{L}_\text{recon} + \beta \cdot \mathcal{L}_\text{KL}$ where $\mathcal{L}_\text{recon}$ is reconstruction loss and $\mathcal{L}_\text{KL} = -\frac{1}{2}\sum(1 + \log \sigma^2 - \mu^2 - \sigma^2)$.

- Train with $\beta \in \{0.5, 1.0, 4.0\}$ in separate runs. β controls the balance between reconstruction and disentanglement. Report reconstruction quality and latent separability for each β.

3. **Reconstruction Quality Evaluation:**

   - Compute on the test set for both AE and best VAE: **MSE** (lower is better), and domain-appropriate metrics (**SSIM** for image-like data, **PSNR** for image-like data, correlation for continuous data).
   - Display sample reconstructions: show 6–10 test examples as side-by-side pairs (original, AE reconstruction, VAE reconstruction). Include pixel-wise or element-wise difference maps.
   - Analyze: which types of data points are hardest to reconstruct? What patterns do reconstruction errors exhibit?

4. **Anomaly Detection Application:**

   - If your dataset has a clear normal vs. abnormal class, use it. Otherwise, designate the majority class as "normal" and test on minority class.
   - Train the AE and VAE on **normal data only**. Evaluate on a mixed test set (both normal and abnormal).
   - Compute reconstruction `MSE` for each test example. Use this as an anomaly score: higher reconstruction error $\rightarrow$ more likely to be anomalous.
   - For the VAE, also compute an alternative anomaly score: reconstruction error + KL divergence term.
   - Report `AUC-ROC` for anomaly detection. Compare against Workshop 1 and 2 baselines.

5. **Latent Space Analysis:**

   - Encode all test-set data with the AE encoder and VAE encoder (using μ, not sampled z). Produce `t-SNE` and `UMAP` plots colored by ground-truth labels (if available).
   - Compare with latent representations from the Workshop 2 supervised model (frozen encoder on test data).
   - For visualization, limit to 2D for interpretability. Quantify class separability using **Silhouette Score** (`sklearn.metrics.silhouette_score`) on the latent vectors.
   - Discussion: does the supervised model produce more organized latent representations than unsupervised autoencoders? What does this reveal about the value of label supervision?

6. **Optional: Data Augmentation or Denoising:**

   - Train a denoising autoencoder: add noise to data during training, but train the model to reconstruct clean data.

- Or: use the VAE to sample synthetic data points (by sampling $z \sim \mathcal{N}(0, \mathbf{I})$ and decoding). Augment the training set and retrain a Workshop 2 model. Report if augmentation improves performance.

Deliverables and Documentation

1. **Autoencoder Report (`PDF`):**
   - Motivation: why unsupervised learning for your domain?
   - AE and VAE architecture descriptions
   - Training curves (reconstruction loss per epoch)
   - Reconstruction quality evaluation and sample panels
   - Anomaly detection results and AUC-ROC comparison
   - Latent space visualization (`t-SNE`, `UMAP`) and silhouette scores
   - Analysis: supervised vs. unsupervised representations
2. **Code:**
   - `src/autoencoder.py` — AE and VAE implementations
   - `src/train_ae.py` — training script
   - `src/evaluate_ae.py` — evaluation (reconstruction, anomaly detection)
   - `src/analyze_latent.py` — latent space visualization and analysis
   - `checkpoints/ae_best.pt` — best AE checkpoint
   - `checkpoints/vae_best.pt` — best VAE checkpoint
   - `runs/workshop4/` — training logs

Submission Requirements

- The `PDF` report must follow the naming convention: `groupN_workshop4.pdf`
- Submit through the designated course platform by the specified deadline
- All metric values must be reproducible by running the submitted code with submitted checkpoints
- Latent visualizations must be included in the report (either as embedded figures or referenced image files)

Grading Note: 40% of the workshop grade will be determined by an in-class quiz administered during the course. This quiz will assess your team's understanding of autoencoder and VAE architectures, the reparameterization trick, latent space interpretation, and unsupervised representation learning. The quiz emphasizes mastery of concepts and the ability to explain architectural choices and reconstruction quality analysis.

Important Guidelines and Considerations

- **`KL` Vanishing Problem:** If VAE reconstructions are perfect but the latent space is disorganized, the `KL` term has been overwhelmed (posterior collapse). Increase β or implement `KL` annealing: start with $\beta = 0$ and linearly ramp to target over the first 10 epochs.
- **Choosing Latent Dimension:** Start with $z_$dim approximately $\approx$ 5–10% of the data dimension. For very high-dimensional data, begin smaller and expand if needed. Monitor reconstruction quality vs. latent compression trade-off.
- **Reconstruction Loss Selection:** MSE is standard, but consider alternatives: binary cross-entropy for binary data, negative log-likelihood for probabilistic targets. Match the loss to your data type.
- **Anomaly Detection Requires Care:** The reconstruction-error threshold should be tuned on a validation set, not the test set. High variance in normal reconstructions can lead to false positives.
- **Cumulative Comparison:** Update your results table from Workshops 1–2 with AE/VAE anomaly detection results. This table tracks performance across all methods implemented so far.

Good luck, and enjoy the elegance of unsupervised learning — watching a neural network reconstruct high-dimensional data from a small latent code, discovering structure without any labels, is a striking demonstration of what representation learning can achieve!